



TÀI LIỆU ÔN TẬP PHÒNG VẤN CHUYÊN SÂU: DỰ ÁN EXAMTRUST

Dự án: ExamTrust — Nền tảng Đánh giá Học thuật & Khảo thí Trực tuyến Thông minh (Smart Academic Assessment & Integrity Proctoring Platform)

Tính chất: Khóa luận Tốt nghiệp Kỹ thuật Phần mềm — ĐH Tôn Đức Thắng (Tốt nghiệp loại Giỏi, GPA 8.34/10)

Vai trò: Lead Full-stack Architecture & AI Integration

Mục tiêu phỏng vấn: Vị trí **Fresher / Junior Backend Developer (NestJS/NodeJS)**, **Full-stack Engineer**, hoặc **AI Application / LLM Engineer**.

MỤC LỤC

- [Elevator Pitch — Cách giới thiệu dự án trong 60 - 90 giây.](#)
- [Kiến trúc Tổng thể & Dòng dữ liệu \(System Architecture\).](#)
- [6 Điểm Sáng Kỹ thuật \(Technical Highlights\) Cần Thuộc Lòng.](#)
- [Bộ Câu Hỏi & Câu Trả Lời Phỏng Vấn Thực Chiến \(STAR Model\).](#)
- [Các Câu Hỏi Bẫy của Interviewer & Cách Phản Ứng Khôn Ngoan.](#)
- [Bảng Tra Cứu Thông Số & Số Liệu Định Lượng \(Cheat Sheet\).](#)

1. ELEVATOR PITCH (GIỚI THIỆU DỰ ÁN TRONG 60 - 90 GIÂY)

Kịch bản trả lời mẫu:

"Em xin phép chia sẻ về **ExamTrust**, đây là đề tài Khóa luận Tốt nghiệp chuyên ngành Kỹ thuật Phần mềm của em tại Đại học Tôn Đức Thắng, nơi em đảm nhiệm vai trò Lead Full-stack Architecture và Tích hợp AI.

Vấn đề lớn nhất của khảo thí trực tuyến hiện nay là **nguy cơ lộ đề, gian lận thi cử**, và **hệ thống bị nghẽn I/O khi tích hợp AI xử lý tác vụ nặng**. Để giải quyết bài toán này, em đã thiết kế kiến trúc chuẩn Enterprise phân tách thành 2 tiến trình độc lập:

- API Web Server (NestJS)** phục vụ trả lời HTTP nhanh dưới 50ms cho thí sinh.
- AI Worker độc lập (Bull Queue + Redis)** chuyên xử lý sinh câu hỏi, vector search và vision proctoring mà không làm nghẽn Event Loop.

Hệ thống nổi bật với 3 tính năng cốt lõi:

- **Sinh đề ngẫu nhiên hóa ma trận và Snapshot bất biến**: Khóa cứng trạng thái đề lúc bắt đầu thi, đảm bảo toàn vẹn dữ liệu.

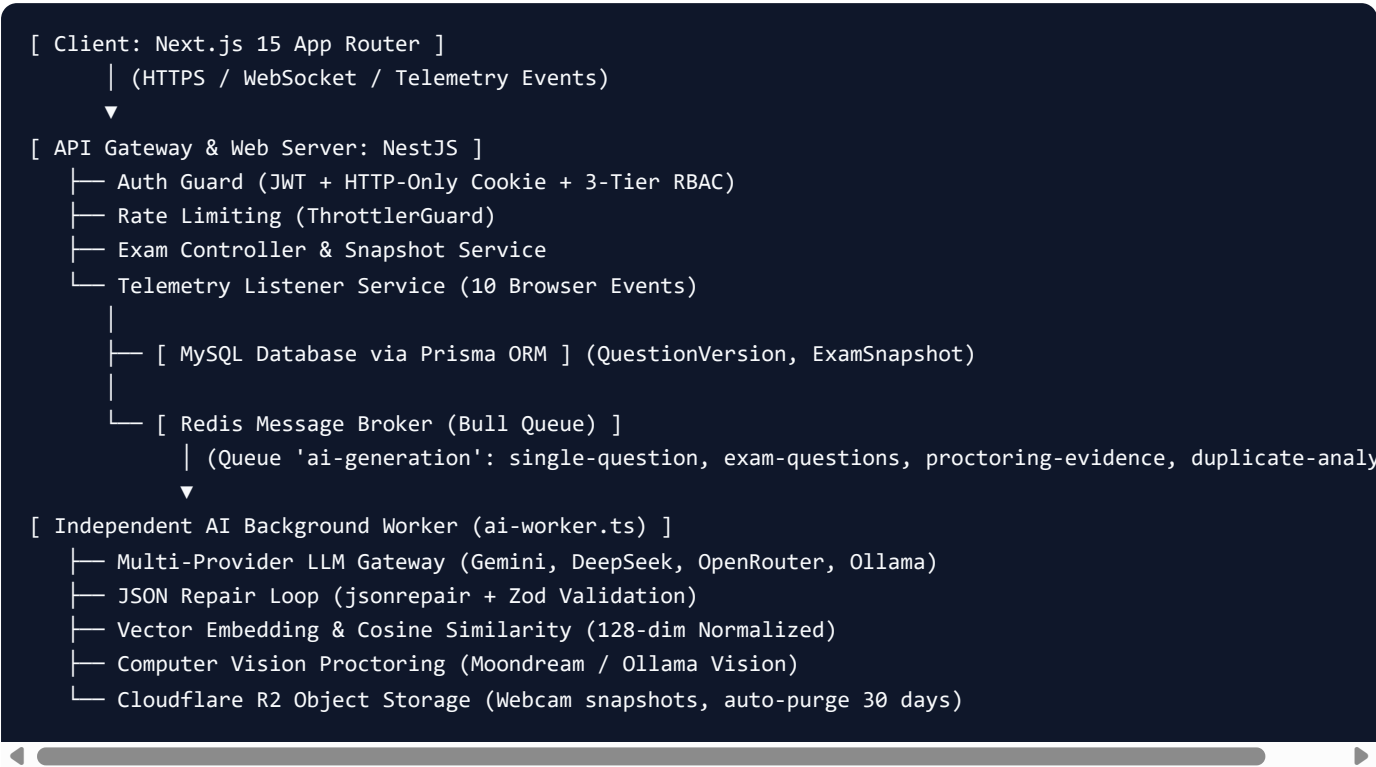
*- **Giám sát liên chính 3 tầng không xâm phạm (Privacy-First)**: Thu thập 10 tín hiệu Browser Telemetry và thị giác máy tính cục bộ, tự xóa sau 30 ngày.*

- **Multi-Provider LLM Gateway với cơ chế JSON Repair tự phục hồi**: Tích hợp 5+ mô hình (Gemini, DeepSeek, Ollama), tự động fallback khi lỗi API.

Dự án được em quản lý nghiêm ngặt qua **28 Pull Requests trên GitHub**, mỗi PR đều đính kèm bằng chứng ảnh chụp và video chạy thực tế trước khi merge."

2. KIẾN TRÚC TỔNG THỂ & DÒNG DỮ LIỆU

Sơ đồ Phân tầng (Architecture Layers)



3. 6 ĐIỂM SÁNG KỸ THUẬT CẦN THUỘC LÒNG

1. Phân tách API Web Server & AI Worker (Distributed Bull Queue)

- Vấn đề:** Các tác vụ AI như **Giảng viên yêu cầu AI sinh câu hỏi nháp / trích xuất câu hỏi từ tài liệu** hoặc **mô hình thị giác máy tính phân tích ảnh chụp webcam phòng thi** tốn từ **3 đến 15 giây**. Nếu xử lý đồng bộ trực tiếp trong HTTP Request của NestJS, Event Loop của Node.js sẽ bị giữ kết nối (holding connection), gây cạn kiệt connection pool và nghẽn toàn bộ HTTP API khi có hàng trăm thí sinh đang thi cử và nộp bài.
(Lưu ý mấu chốt: Nghiệp vụ tạo đề thi trong hệ thống hoàn toàn chạy bằng thuật toán bốc câu hỏi từ ngân hàng câu hỏi chỉ mất vài chục mili-giây, không hề đợi AI).
- Giải pháp:**
 - Khi Giảng viên yêu cầu AI sinh câu hỏi hoặc client gửi ảnh webcam kiểm tra, API Server đẩy payload vào **Redis Bull Queue (queue 'ai-generation')** và trả ngay HTTP 202 Accepted kèm `jobId`.
 - Tiến trình `ai-worker.ts` chạy độc lập, lắng nghe job (`task: 'single-question'` , `task: 'exam-questions'` , `task: 'proctoring-evidence'`), gọi AI Gateway, chạy JSON Repair Loop và ghi kết quả vào database.
 - Phía client polling hoặc nhận thông báo trạng thái qua WebSocket/SSE khi job hoàn tất.
- Kết quả:** HTTP API của Web Server luôn phản hồi tức thì (< 50ms), luồng làm bài và nộp bài của thí sinh hoàn toàn không bị ảnh hưởng bởi tải xử lý AI.

2. Nguyên tắc Human-in-the-Loop & Snapshot Đề thi Bất biến (Immutable Exam State)

- **Triết lý Thiết kế — Tuyệt đối không để AI tự ý sinh cả đề thi lung tung:**
 - Trong khảo thí học thuật, việc cho LLM tự động sinh hoàn chỉnh cả một đề thi rồi phát trực tiếp cho thí sinh là điều cấm kỵ vì rủi ro ảo giác (hallucination) và sai lệch chuẩn đào tạo.
 - Do đó, trong ExamTrust: AI **CHỈ đóng vai trò hỗ trợ sinh câu hỏi gợi ý / thảo thảo câu hỏi (Question Drafts)**. Giảng viên bắt buộc phải kiểm duyệt (review), biên tập lại nội dung, đáp án, độ khó trước khi lưu thành câu hỏi chính thức trong Ngân hàng câu hỏi.
- **Cơ chế Sinh đề & Snapshot Đề thi Bất biến:**
 - Đề thi chỉ được cấu thành từ Ngân hàng câu hỏi đã qua kiểm duyệt (theo danh sách câu hỏi Giảng viên chọn hoặc bốc ngẫu nhiên theo ma trận độ khó Dễ/Trung bình/Khó bằng SQL query).
 - Khi thí sinh bấm "Bắt đầu làm bài", hệ thống ngay lập tức serialize toàn bộ nội dung câu hỏi và thứ tự đáp án (shuffled) thành một bản ghi **Snapshot đóng băng** trong database (`ExamSessionSnapshot`).
 - Trong suốt thời gian làm bài, thí sinh chỉ đọc và tương tác với bản Snapshot này. Dù giảng viên có chỉnh sửa ngân hàng câu hỏi hay thí sinh reload lại trang thì đề thi vẫn giữ nguyên vẹn 100%.

3. Ngân hàng Câu hỏi Git-Like Versioning (`QuestionVersion`)

- **Vấn đề:** Đề thi cần kiểm toán (Audit Trail) phục vụ thanh tra đào tạo. Nếu chỉ cập nhật trực tiếp dòng dữ liệu trong DB, ta sẽ mất dấu ai đã sửa nội dung gì và không thể khôi phục phiên bản trước.
- **Giải pháp:** Thiết kế bảng `Question` quan hệ 1-N với `QuestionVersion` . Mỗi lần giảng viên chỉnh sửa, hệ thống tạo bản ghi version mới lưu diff (thay đổi), trạng thái duyệt, thời gian và ID người sửa. Hỗ trợ Rollback 1-click về phiên bản cũ.

4. Telemetry Phòng thi 10 Tín hiệu Không Xâm phạm (Privacy-First)

- **Vấn đề:** Nhiều phần mềm thi cử ép sinh viên cài driver can thiệp kernel/OS hoặc chiếm quyền camera liên tục, gây lo ngại rò rỉ quyền riêng tư và dễ bị antivirus chặn.
- **Giải pháp:** Thu thập 10 sự kiện hoàn toàn qua Browser Standard APIs:
 1. `blur` : Mất focus khỏi cửa sổ làm bài.
 2. `visibilitychange` : Chuyển tab hoặc minimize browser.
 3. `fullscreenchange` : Thoát chế độ toàn màn hình.
 4. `copy / cut` : Cố tình sao chép đề bài.
 5. `paste` : Dán nội dung bên ngoài vào bài thi.
 6. `contextmenu` : Chuột phải để inspect hoặc search Google.
 7. `keydown (F12, Ctrl+Shift+I, Alt+Tab)` : Phím tắt mở DevTools hoặc chuyển ứng dụng.
 8. `resize` : Thu nhỏ cửa sổ bất thường.
 9. `beforeunload` : Cố tình tắt trình duyệt giữa chừng.
 10. `multi-display detect` : Phát hiện gắn thêm màn hình phụ.
- **Xử lý:** Tổng hợp thành điểm rủi ro **Risk Score (0 - 100)** theo thời gian thực và lưu audit log.

5. Multi-Provider LLM Gateway & JSON Repair Loop

- **Vấn đề:** Các API LLM đám mây (Gemini, DeepSeek) thường gặp lỗi: Rate limit (HTTP 429), server timeout (504), hoặc trả về JSON bị lỗi cú pháp (thiếu ngoặc nhọn, lẫn markdown fence ````json`).
- **Giải pháp:**
 - **Trừu tượng hóa Interface** `LlmProvider`: Đóng gói Gemini, DeepSeek, OpenRouter, NVIDIA, Local Ollama. Nếu Provider chính fail sau 2 lần retry, hệ thống tự động fallback sang Provider dự phòng.
 - **Vòng lặp JSON Repair:** Sau khi nhận string từ LLM, loại bỏ markdown block, đưa qua thư viện `jsonrepair` để vá tự động các lỗi cú pháp JSON phổ biến, rồi validate lại qua **Zod schema**.
- **Kết quả:** Đạt tỷ lệ phân tích cú pháp JSON thành công **100%**, không bị crash tiến trình.

6. Khung Benchmark Tự động LLM-as-a-Judge (`golden-dataset.ts`)

- **Vấn đề:** Làm sao biết prompt mới tốt hơn prompt cũ? Làm sao kiểm soát ảo giác (hallucination) và đảm bảo LLM sinh câu hỏi gợi ý đúng chủ đề và độ khó yêu cầu?
- **Giải pháp:**
 - Xây dựng bộ `golden-dataset.ts` chứa 50+ ca kiểm thử mẫu có đáp án đối chuẩn.
 - Module `AiEvaluationJudge` cho một mô hình lớn (Gemini 1.5 Pro) chấm điểm chất lượng câu hỏi sinh ra bởi mô hình worker (Ollama/DeepSeek) theo thang Rubric: Độ chính xác học thuật, độ phân hóa câu hỏi, tính rõ ràng và tuân thủ schema.

4. BỘ CÂU HỎI & CÂU TRẢ LỜI PHÒNG VẤN THỰC CHIẾN

? Câu 1: Em hãy giải thích lý do tại sao em chọn tách biệt API Web Server và Worker Process qua Redis Bull Queue mà không dùng trực tiếp Promise/Async trong NestJS?

- **Gợi ý trả lời:**

"Node.js hoạt động dựa trên cơ chế Single-Threaded Event Loop. Khi một async operation như gọi API LLM sinh câu hỏi từ tài liệu hoặc phân tích ảnh webcam diễn ra, nó giữ kết nối HTTP TCP kết nối tới server trong hàng chục giây.

Nếu em chỉ dùng `async/await` nội bộ trong Controller:

1. Nếu server bị restart hoặc crash giữa chừng, toàn bộ các tác vụ đang sinh câu hỏi hoặc phân tích ảnh dở dang sẽ bị mất vĩnh viễn (in-memory loss).

2. Khi chịu tải cao (nhiều giảng viên đồng thời yêu cầu AI sinh bộ câu hỏi hoặc nhiều ảnh webcam cùng đẩy lên), số lượng concurrent request giữ kết nối quá lâu sẽ làm cạn kiệt connection pool của Node.js/OS, gây nghẽn luôn các request làm bài thi và nộp bài của hàng trăm thí sinh.

(Lưu ý: Nghiệp vụ tạo đề thi trong hệ thống hoàn toàn chạy bằng thuật toán bốc câu hỏi từ ngân hàng câu hỏi chỉ mất vài chục mili-giây, không hề đợi AI).

Vì vậy, em dùng **Redis Bull Queue** để tách thành Background Job: API Server chỉ nhận yêu cầu và đẩy vào Redis (mất < 10ms), trả về `jobId` cho client. Tiến trình `ai-worker.ts` xử lý hàng đợi theo cơ chế concurrency kiểm soát được (ví dụ chỉ chạy 3-5 jobs song song để tránh vượt rate-limit LLM), có cơ chế tự retry với exponential backoff khi mạng chập chờn và lưu trạng thái bền vững vào Redis."

? **Câu 2: Giả sử trong phòng thi có 500 sinh viên cùng nộp bài cùng một thời điểm (kết thúc giờ làm bài), hệ thống của em xử lý áp lực ghi vào Database như thế nào để không bị Deadlock hoặc sập DB?**

- **Gợi ý trả lời:**

"Đây là bài toán giờ cao điểm kinh điển của hệ thống thi cử (Thundering Herd Problem). Em đã tính toán kiến trúc với 3 tầng giảm tải:

*1. **Auto-save định kỳ (Debounced Auto-save):** Trong suốt quá trình làm bài, mỗi khi sinh viên tick chọn đáp án, client tự động gửi update đáp án câu đó sau mỗi 2 giây debounce. Nghĩa là 95% dữ liệu bài làm đã được lưu rải rác từ trước, thời điểm bấm 'Nộp bài' chỉ là cập nhật cờ `isSubmitted = true` và `submittedAt`.*

2. **Payload tối giản:** Khi nộp bài, client chỉ gửi hash kiểm tra tính toàn vẹn và mảng ID đáp án nhỏ gọn thay vì gửi toàn bộ đề thi.

3. **Transaction ngắn & Connection Pooling:** Sử dụng Prisma transaction với timeout nghiêm ngặt, chỉ cập nhật trạng thái session mà không thực hiện chấm điểm ngay trong request đó. Tác vụ chấm điểm tự luận bằng AI được đẩy tiếp vào Bull Queue để chấm bất đồng bộ phía sau."

? **Câu 3: Làm thế nào em đảm bảo LLM trả về đúng định dạng JSON mà hệ thống Backend có thể parse được 100% không bị crash?**

- **Gợi ý trả lời:**

"Mô hình ngôn ngữ lớn bản chất là sinh xác suất từ tiếp theo, nên không có gì bảo đảm nó luôn trả về JSON hợp lệ nếu chỉ dùng prompt chay. Em áp dụng chiến lược phòng thủ 4 lớp:

1. **System Prompt & JSON Schema:** Ép mô hình ở tầng system prompt chỉ trả về raw JSON, không kèm lời chào hay markdown.

2. **Trích xuất chuỗi Regex:** Bóc tách chuỗi nằm giữa `{...}` hoặc `[...]` để loại bỏ các đoạn văn bản thừa hoặc markdown fence ````json`.*

*3. **Vòng lặp jsonrepair**: Sử dụng thư viện `jsonrepair` để tự động vá các lỗi cú pháp LLM hay mắc phải như: thiếu dấu phẩy, dùng nháy đơn thay cho nháy kép, trailing comma, hoặc chuỗi chưa đóng ngoặc nhọn do chạm max tokens.

4. **Zod Validation & Fallback:** Đưa JSON đã parse qua Zod Schema. Nếu Zod báo thiếu trường bắt buộc, worker tự động trigger retry với nhiệt độ (temperature) thấp hơn (0.1) hoặc chuyển sang model khác.

Nhờ pipeline này, em đạt tỷ lệ parse thành công 100% trên tập dữ liệu kiểm thử."

? **Câu 4: Em xử lý vấn đề Cheating Detection (chống gian lận) như thế nào mà không cần bắt sinh viên cài ứng dụng can thiệp sâu vào máy tính?**

- **Gợi ý trả lời:**

"Em lựa chọn triết lý **Privacy-first Proctoring**. Thay vì cài phần mềm can thiệp sâu vào kernel OS gây lo ngại rò rỉ quyền riêng tư và dễ bị hệ điều hành chặn, em khai thác triệt để các **Browser Event APIs** kết hợp AI phân tích ảnh webcam chu kỳ:

- **Tầng 1 - Tín hiệu trình duyệt:** Bắt sự kiện `visibilitychange` và `window.blur` để biết khi thí sinh đổi tab hoặc mở ứng dụng khác; chặn và log lại thao tác `copy`, `paste`, phím tắt DevTools `F12`.

- **Tầng 2 - Thị giác máy tính đa phương thức:** Định kỳ chụp ảnh webcam độ phân giải thấp gửi về Cloudflare R2, dùng mô hình vision (Moondream/Ollama Vision) chạy offline để gán nhãn 9 nguy cơ: vắng mặt, nhiều người trước màn hình, sử dụng điện thoại.

- **Tầng 3 - Dual-layer Risk Scoring & Audit Trail:** Hệ thống chấm điểm rủi ro từ 0 đến 100. AI **hoàn toàn không tự ý đánh trượt sinh viên** mà chỉ cung cấp bằng chứng timestamp cho giảng viên xem xét lại (Human-in-the-Loop). Đồng thời toàn bộ ảnh webcam được cấu hình tự động xóa sau 30 ngày."

? Câu 5: Trong dự án ExamTrust, em đã áp dụng quy trình Git Flow và Pull Request như thế nào? Tại sao em lại có tới 28 Pull Requests chuẩn hóa?

• Gợi ý trả lời:

"Ngay từ đầu khóa luận, em đặt mục tiêu phát triển sản phẩm với kỷ luật như một dự án Enterprise thật sự, không làm theo kiểu 'code một lèo commit thẳng vào main'.

Quy trình em tuân thủ gồm:

1. **Phân nhánh Feature Branch:** Mọi tính năng (như `feature/ai-worker-bullmq`, `feature/telemetry-monitor`, `feature/exam-snapshot`) đều xuất phát từ nhánh `develop`.

2. **Quy chuẩn PR Proof of Work:** Trong mỗi Pull Request trên GitHub, em bắt buộc phải có:

- Mô tả rõ ràng Scope of Changes.*
- Báo cáo kết quả kiểm tra tĩnh (`tsc -b` pass 0 lỗi).*
- **Đính kèm ảnh chụp màn hình UI hoặc video quay màn hình chạy thực tế** làm minh chứng.*
Tổng cộng qua toàn bộ quá trình phát triển, em đã hoàn thành **28 Pull Requests chuẩn hóa** được review và merge sạch sẽ. Điều này giúp lịch sử commit của dự án vô cùng rõ ràng, dễ dàng truy vết và không bao giờ bị xung đột mã nguồn lớn."

5. CÁC CÂU HỎI BẦY CỦA INTERVIEWER & CÁCH PHẢN ỨNG

⚠ Bầy 1: "Em dùng AI nhiều như vậy (Gemini, DeepSeek, Cursor...), có phải toàn bộ code của em là do AI viết giùm không?"

• Cách trả lời tự tin:

"Dạ thưa anh/chị, AI là đòn bẩy gia tăng tốc độ gõ mã nguồn của em lên gấp 3 đến 5 lần, nhưng **toàn bộ tư duy kiến trúc, thiết kế database và quy tắc nghiệp vụ là do em trực tiếp làm chủ**.

Nếu giao phó toàn bộ cho AI mà không có nền tảng học thuật vững (em tốt nghiệp loại Giỏi chuyên ngành Kỹ thuật Phần mềm TDTU, GPA 8.34), AI sẽ sinh ra code rời rạc, xung đột schema, không xử lý được race conditions trong database và không thể thiết kế được một pipeline phân tán ổn định như Redis Bull Queue.

Em đóng vai trò là **Architect-in-the-Loop**: Em thiết kế data model, định nghĩa interface contracts, thiết lập các ràng buộc bảo mật RBAC; AI chỉ là trợ lý thực thi thần tốc. Mọi dòng code merge vào 28 PRs đều qua khâu kiểm chứng kiểu tĩnh và chạy thử thực tế của em."

⚠ Bẫy 2: "Tại sao em dùng cả Prisma ORM lẫn MySQL, tại sao không dùng MongoDB cho linh hoạt khi lưu trữ câu hỏi và bài thi?"

• Cách trả lời chuẩn kỹ thuật:

"Đối với hệ thống khảo thí và chấm điểm, tính chất **ACID (Atomicity, Consistency, Isolation, Durability)** và tính toàn vẹn quan hệ là tối quan trọng. Ví dụ: một bài thi bắt buộc phải gắn liền với sinh viên hợp lệ, câu hỏi thuộc về đề thi, và số điểm không được phép sai lệch do race condition.

Nếu dùng NoSQL (MongoDB), ta sẽ dễ rơi vào tình trạng dữ liệu không đồng nhất (data inconsistency) khi cập nhật đồng thời nhiều quan hệ. Với MySQL kết hợp Prisma ORM, em vừa có được **Foreign Key Constraints** chặt chẽ, vừa có **Strict TypeScript Types** đồng bộ tự động từ schema sang code backend, giúp giảm thiểu 90% lỗi sai kiểu dữ liệu ở compile-time."

6. BẢNG TRA CỨU THÔNG SỐ ĐỊNH LƯỢNG (CHEAT SHEET)

Hạng mục kỹ thuật	Thông số thực tế trong ExamTrust	Ý nghĩa thuyết phục nhà tuyển dụng
Pull Requests	28 Closed PRs trên GitHub	Kỷ luật làm việc nhóm, chuẩn hóa Git Flow
QPS Response	< 50ms cho HTTP endpoints	Nhờ đẩy tác vụ nặng sang Background Worker
Dung sai JSON	100% Parse Success	Áp dụng jsonrepair + AbortSignal + Zod
Browser Telemetry	10 Event Types	Giám sát không xâm phạm quyền riêng tư
Multi-Provider AI	5+ Providers (Gemini, DeepSeek, Ollama...)	Chống phụ thuộc 1 vendor, runtime fallback
Chi phí Hạ tầng	\$0/tháng (Cloudflare Serverless)	Tối ưu hóa chi phí cho môi trường học thuật
Học vấn	Tốt nghiệp Loại Giỏi (GPA 8.34/10)	Nền tảng thuật toán, phần mềm bài bản